

Chapter 11 Inheritance and Polymorphism

(Polymorphism)



Objectives

- To discover **polymorphism** and **dynamic binding** (§§11.7–11.8).
- To describe **casting** and explain why explicit downcasting is necessary (§11.9).
- To explore the **equals** method in the **Object** class (§11.10).
- To store, retrieve, and manipulate objects in an **ArrayList** (§11.11).
- To implement a Stack class using ArrayList (§11.12).



The Pillars of OOP

The Four Pillars



Polymorphism

- **Polymorphism** is a Greek word; (poly) means *many* and (morph) means *shapes*.
- The term **Polymorphism** literally means "having many forms".
- Polymorphism means that *a variable of a supertype can refer to a subtype object.*



Polymorphism

A class defines a type.

A type defined by a *subclass* is called a *subtype*.

A type defined by its *superclass* is called a *supertype*.

Therefore, you can say that **Circle** is a *subtype* of **Shape** and **Shape** is a *supertype* for **Circle**.



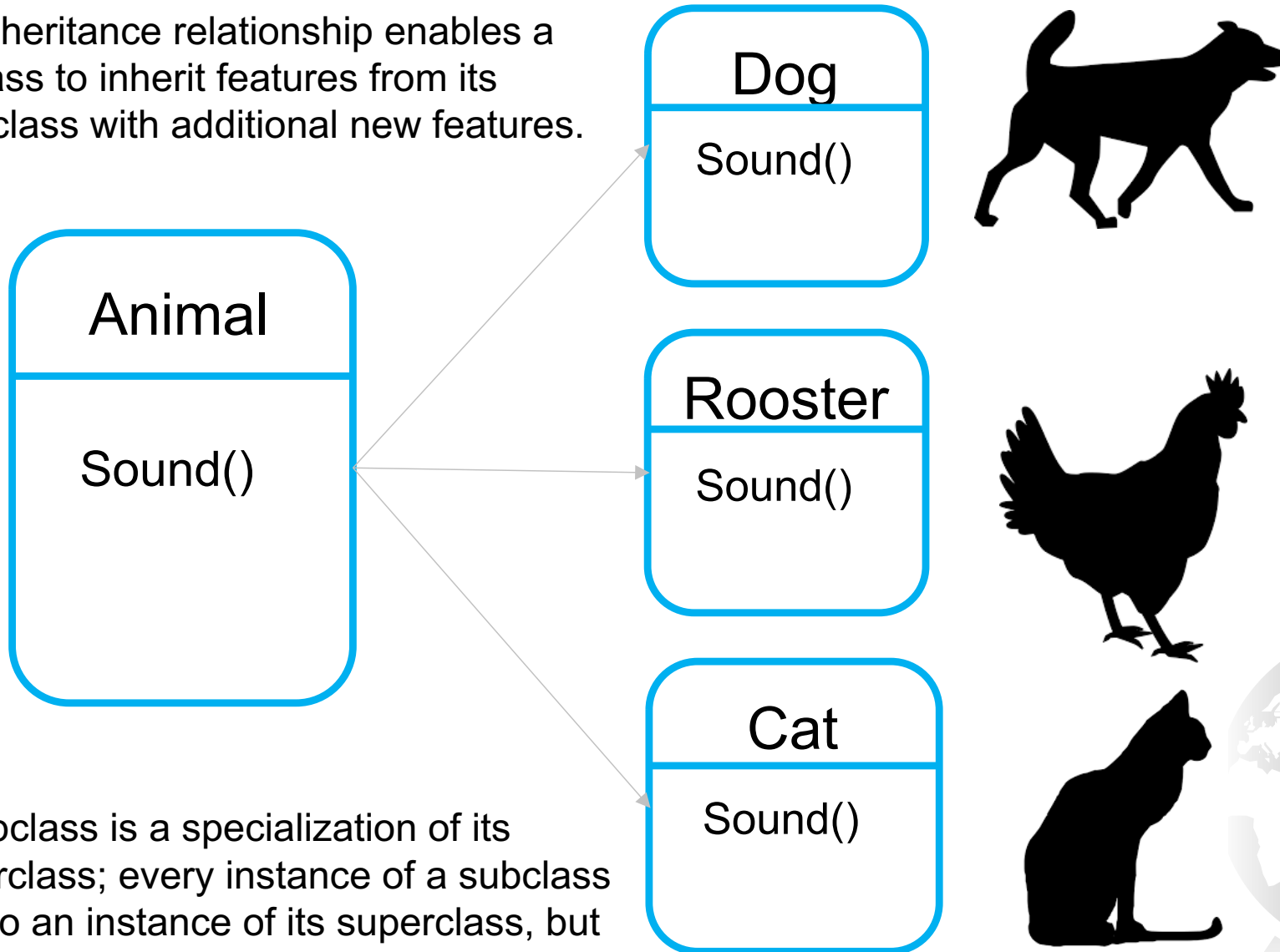
PolymorphismDemo

Run



Animals make sounds

The inheritance relationship enables a subclass to inherit features from its superclass with additional new features.

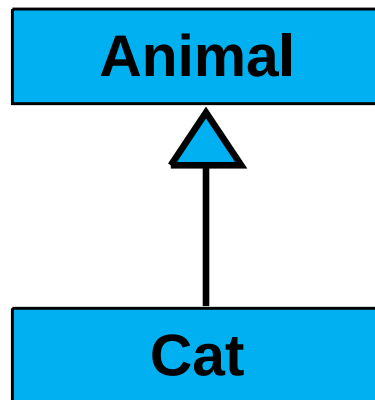


A subclass is a specialization of its superclass; every instance of a subclass is also an instance of its superclass, but not vice versa.



References and Inheritance

- The `Animal` class is used to derive a class called `Cat`. A cat is an animal. Therefore, `Animal` reference could be used to point to a `Cat` object.



```
Animal animal;  
animal = new Cat();
```

- All Animals make sounds. But each one has its own sound.



Declared type and Actual type

- A variable must be declared a type.
`Object o = new GeometricObject();`
o's declared type is **Object**.
- A variable of a reference type can hold a null value or a reference to an instance of the declared type.
- The actual type of the variable is the actual class for the object referenced by the variable.

`Object o = new GeometricObject();`
o's actual type is **GeometricObject**.



References and Inheritance

- An object reference can refer to an object of its class, or to an object of any class related to it by inheritance.

```
Animal animal = new Cat();  
animal.sound();  
animal = new Duck();  
animal.sound();
```

A ***polymorphic reference*** is a variable that can refer to different types of objects at different points in time.



References and Inheritance

```
Animal animal = new Cat();
```

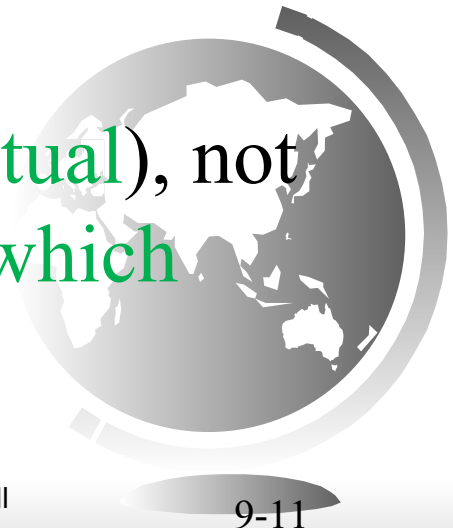
```
animal.sound();
```

```
animal = new Duck();
```

```
animal.sound();
```

The method invoked through a polymorphic reference can change from one invocation to the next.

It is the type of the object being referenced (**actual**), not the reference (**declared**) type, that **determines which method is invoked**.



Polymorphic call

- The inheritance relationship enables a subclass to inherit features from its superclass with additional new features.
- A subclass is a specialization of its superclass; every instance of a subclass is also an instance of its superclass, but not vice versa.
- For example, every circle is a geometric object, but not every geometric object is a circle.
- Therefore, you **can always pass an instance of a subclass to a parameter of its superclass type.**



```
public class PolymorphismDemo {
```

```
/** Main method */
```

```
public static void main(String[] args) {
```

```
// Display circle and rectangle properties
```

```
displayObject(new Circle (1, "red", false));
```

```
displayObject(new Rectangle (1, 1, "black", true));
```

```
}
```

Polymorphic
call

An object of a subclass can be used wherever its
superclass object is used.

```
/** Display geometric object properties */
```

```
public static void displayObject(Shape object)
```

```
{
```

```
    System.out.println("Created on " + object.getDateCreated() +  
                        ". Color is " + object.getColor());
```

```
}
```

```
}
```



```
public class PolymorphismDemo {  
    /** Main method */  
    public static void main(String[] args) {
```

```
        // Display circle and rectangle properties  
        displayObject(new Circle (1, "red", false));  
        displayObject(new Rectangle (1, 1, "black", true));  
    }
```

Polymorphic
call

An object of a subclass can be used wherever its
superclass object is used.

```
    /** Display geometric object properties */  
    public static void displayObject(Shape object)  
    {  
        System.out.println("Created on " + object.getDateCreated() +  
                           ". Color is " + object.getColor());  
    }  
}
```

```
Created on Mon Mar 09 19:25:20 EDT 2011. Color is red  
Created on Mon Mar 09 19:25:20 EDT 2011. Color is black
```





- Suppose the `Mammal` class has a method called `run`, and the `Horse` class overrides it.
- Now consider the following invocation:

```
animal.run( );
```

Which run version will be invoked?



Polymorphism

```
class Person {
    public String toString() {
        return "Person";
    }
}
//*****
class Student extends Person {
    public String toString() {
        return "Student";
    }
}
//*****
class GraduateStudent
    extends Student {
}
```

```
public class PolymorphismDemo {
    public static void
        main(String[] args) {

        m(new GraduateStudent());
        m(new Student());
        m(new Person());
        m(new Object());
    }

    public static void m(Object x) {
        System.out.println(x.toString());
    }
}
```

Method m takes a parameter of the Object type. You can invoke it with any object.



Polymorphism

```
class Person {  
    public String toString() {  
        return "Person";  
    }  
}  
//*****  
class Student extends Person {  
    public String toString() {  
        return "Student";  
    }  
}  
//*****  
class GraduateStudent  
    extends Student {  
}
```

```
public class PolymorphismDemo {  
    public static void  
        main(String[] args) {  
  
        m(new GraduateStudent());  
        m(new Student());  
        m(new Person());  
        m(new Object());  
    }  
  
    public static void m(Object x) {  
        System.out.println(x.toString());  
    }  
}
```

This feature is known as *polymorphism*.

Object x = new GraduateStudent();

Object x = new Student();

Object x = new Person();

Object x = new Object();



Generic Programming

```
class Person {  
    public String toString() {  
        return "Person";  
    }  
}  
//*****  
class Student extends Person {  
    public String toString() {  
        return "Student";  
    }  
}  
//*****  
class GraduateStudent  
    extends Student {  
}
```

```
public class PolymorphismDemo {  
    public static void  
        main(String[] args) {  
  
        m(new GraduateStudent());  
        m(new Student());  
        m(new Person());  
        m(new Object());  
    }  
  
    public static void m(Object x) {  
        System.out.println(x.toString());  
    }  
}
```

Polymorphism allows methods to be used generically for a wide range of object arguments. This is known as generic programming.



Dynamic Binding

```
class Person {  
    public String toString() {  
        return "Person";  
    }  
}  
//*****  
class Student extends Person {  
    public String toString() {  
        return "Student";  
    }  
}  
//*****  
class GraduateStudent  
    extends Student {  
}
```

```
public class PolymorphismDemo {  
    public static void  
        main(String[] args) {  
  
        m(new GraduateStudent());  
        m(new Student());  
        m(new Person());  
        m(new Object());  
    }  
  
    public static void m(Object x) {  
        System.out.println(x.toString());  
    }  
}
```

Which `toString()` method will be invoked?

Which implementation is used will be determined dynamically by the Java Virtual Machine at runtime.

This capability is known as *dynamic binding*.



Dynamic Binding

```
class Person {
    public String toString() {
        return "Person";
    }
}
//*****
class Student extends Person {
    public String toString() {
        return "Student";
    }
}
//*****
class GraduateStudent
    extends Student {
}
```

```
public class PolymorphismDemo {
    public static void
        main(String[] args) {

        m(new GraduateStudent());
        m(new Student());
        m(new Person());
        m(new Object());
    }

    public static void m(Object x) {
        System.out.println(x.toString());
    }
}
```

Which `toString()` method will be invoked?

Which implementation is used will be determined dynamically by the Java Virtual Machine at runtime.

This capability is known as *dynamic binding*.



```
Student
Student
Person
Java.lang.Object@130c19b
```

Dynamic Binding

- A method can be implemented in several classes along the inheritance chain. The JVM decides which method is invoked at runtime.



Dynamic Binding

- A method can be defined in a superclass and overridden in its subclass.
- For example, the `toString()` method is defined in the `Object` class and overridden in `GeometricObject`.
- Consider the following code:

```
Object o = new GeometricObject();  
System.out.println(o.toString());
```
- Which `toString()` method is invoked by `o`?



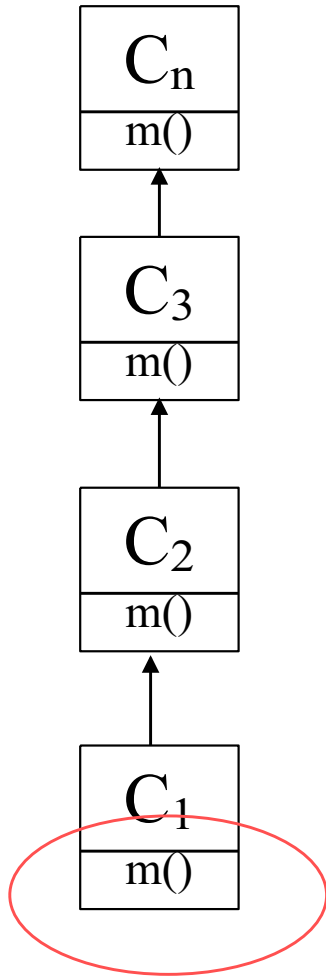


Dynamic Binding

Suppose:

```
C1 o = new C1();  
o.m();
```

Which m will be implemented?



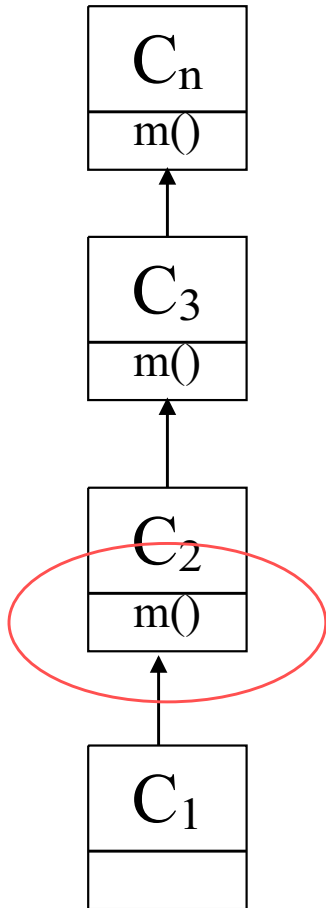


Dynamic Binding

Suppose:

```
C1 o = new C1();  
o.m();
```

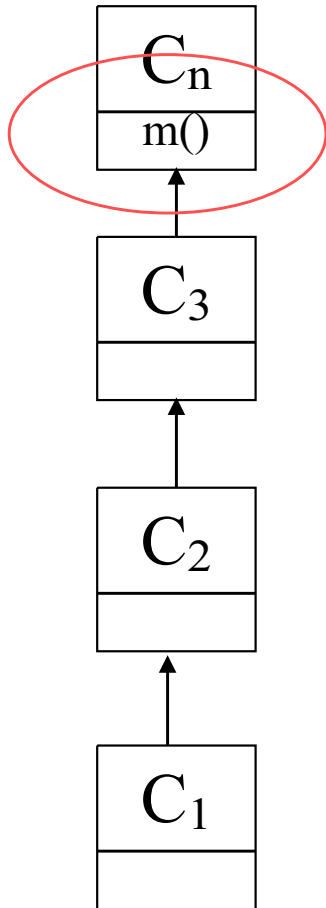
Which m will be implemented?





Dynamic Binding

Suppose:



```
C1 o = new C1();  
o.m();
```

Which m will be implemented?



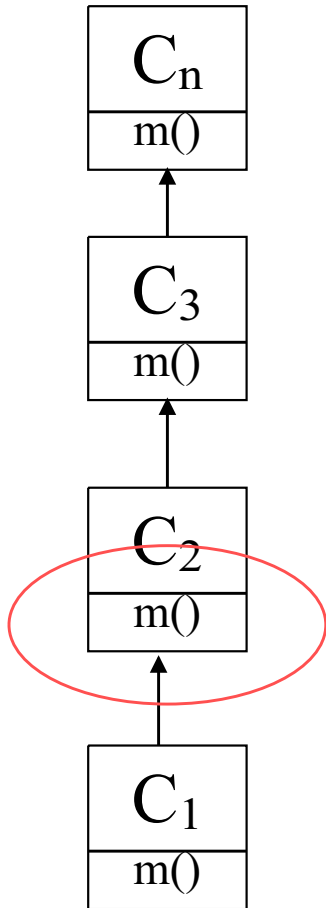


Dynamic Binding

Suppose:

```
C3 o = new C2();  
o.m();
```

Which m will be implemented?



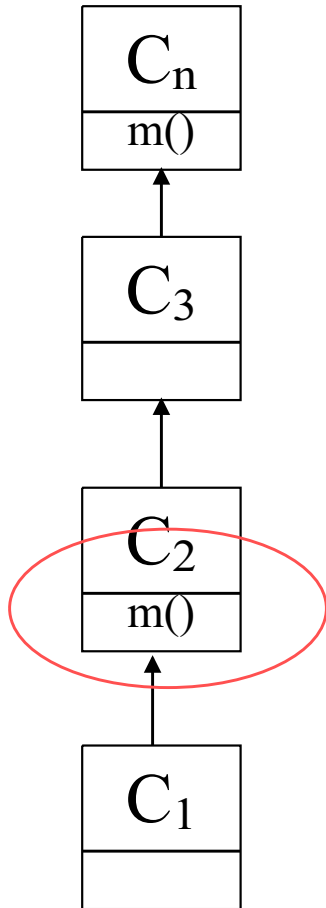


Dynamic Binding

Suppose:

```
C3 o = new C1();  
o.m();
```

Which m will be implemented?



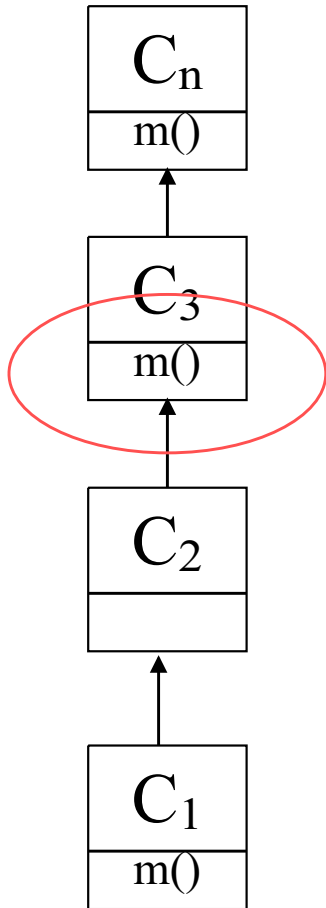


Dynamic Binding

Suppose:

```
C3 o = new C2();  
o.m();
```

Which m will be implemented?



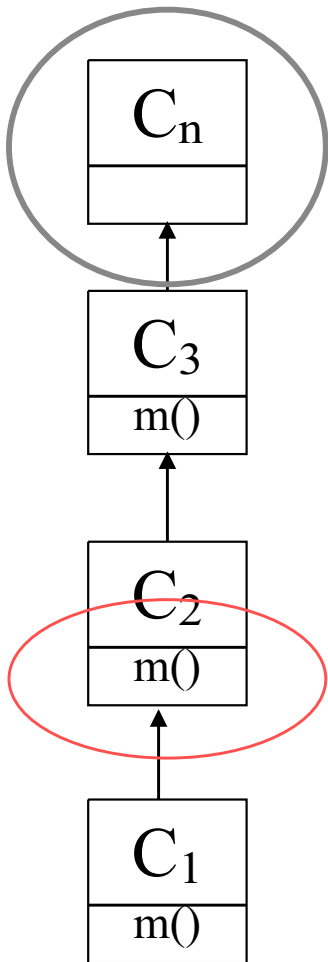


Dynamic Binding

Suppose:

C_3 o = new $C_2()$;
 $o.m()$;

Which m will be implemented?





Dynamic Binding

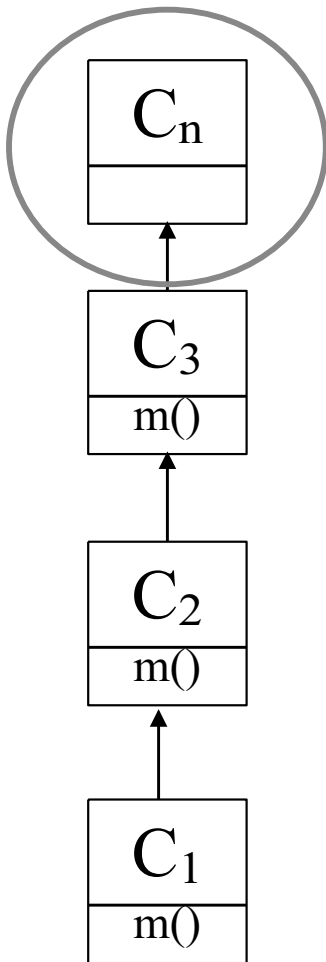
Suppose:

```
Cn o = new C2();  
o.m();
```

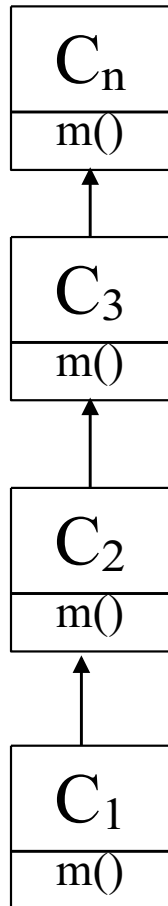
Which m will be implemented?

Error

Declared type should have the invoked method.



Dynamic Binding



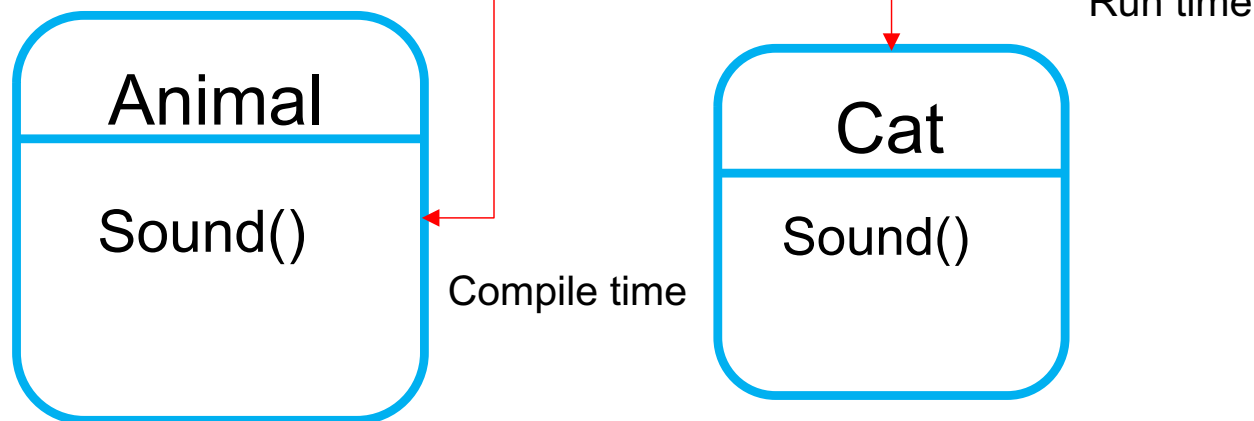
the JVM searches the implementation for the method m in C_1 , C_2 , ..., C_{n-1} and C_n , in this order, until it is found. Once an implementation is found, the search stops, and the first-found implementation is invoked.



Method Matching vs. Binding

- Matching a method signature and binding a method implementation are two issues.
- The declared type of the reference variable decides which method to match at compile time.

```
Animal animal = new Cat();  
animal.sound();
```



Method Matching vs. Binding

- The compiler finds a matching method according to **parameter type**, **number of parameters**, and **order of the parameters at compilation time**.

methodA(int a, char b)

- A method may be implemented in several subclasses.
- The Java Virtual Machine dynamically binds the implementation of the method at runtime.



What is wrong in the following code?

```
1 public class Test {  
2     public static void main(String[] args) {  
3         Integer[] list1 = {12, 24, 55, 1};  
4         Double[] list2 = {12.4, 24.0, 55.2, 1.0};  
5         int[] list3 = {1, 2, 3};  
6         printArray(list1);  
7         printArray(list2);  
8         printArray(list3);  
9     }  
10  
11     public static void printArray(Object[] list) {  
12         for (Object o: list)  
13             System.out.print(o + " ");  
14         System.out.println();  
15     }  
16 }
```



What is wrong in the following code?

```
1 public class Test {
2     public static void main(String[] args) {
3         Integer[] list1 = {12, 24, 55, 1};
4         Double[] list2 = {12.4, 24.0, 55.2, 1.0};
5         int[] list3 = {1, 2, 3};
6         printArray(list1);
7         printArray(list2);
8         printArray(list3);
9     }
10
11     public static void printArray(Object[] list) {
12         for (Object o: list)
13             System.out.print(o + " ");
14         System.out.println();
15     }
16 }
```

int[] cannot be passed to Object[].





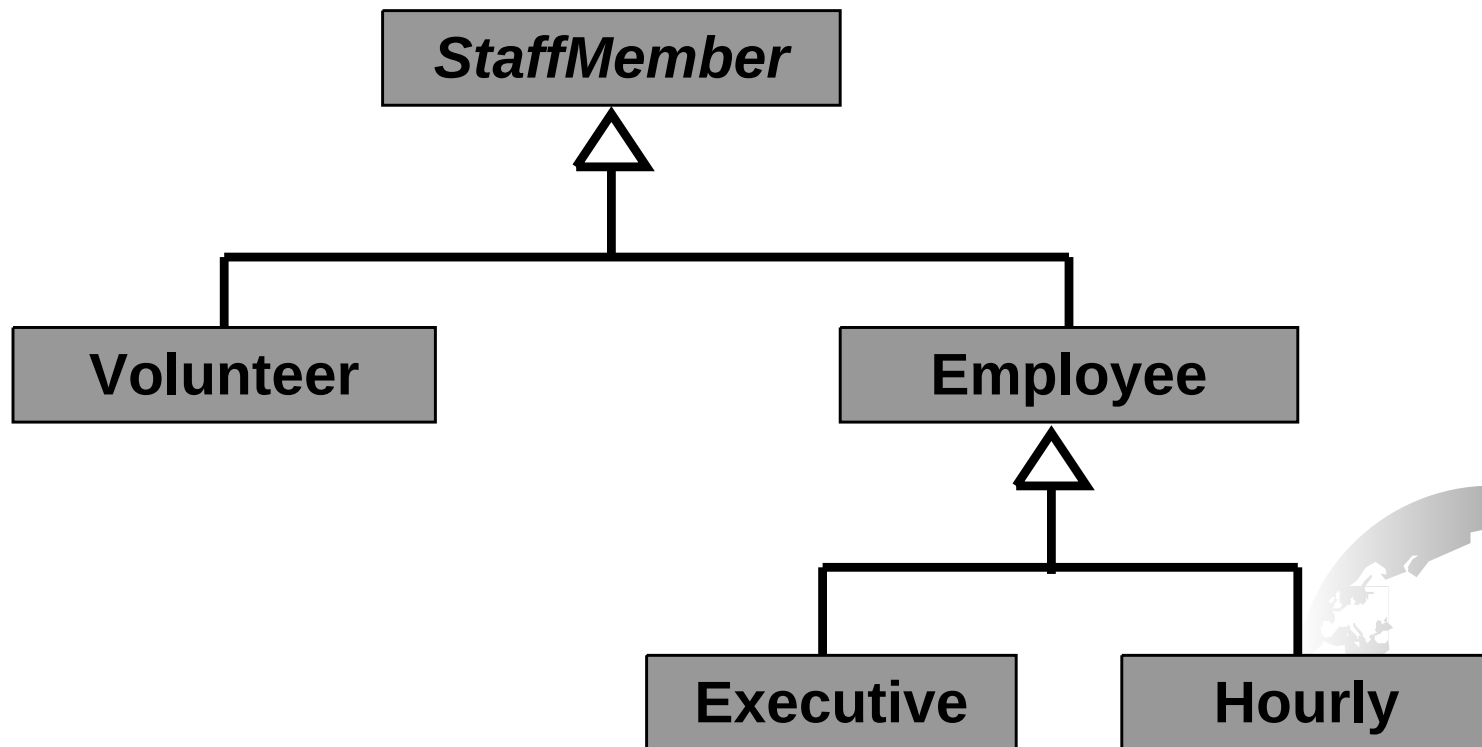
Polymorphism Exercise

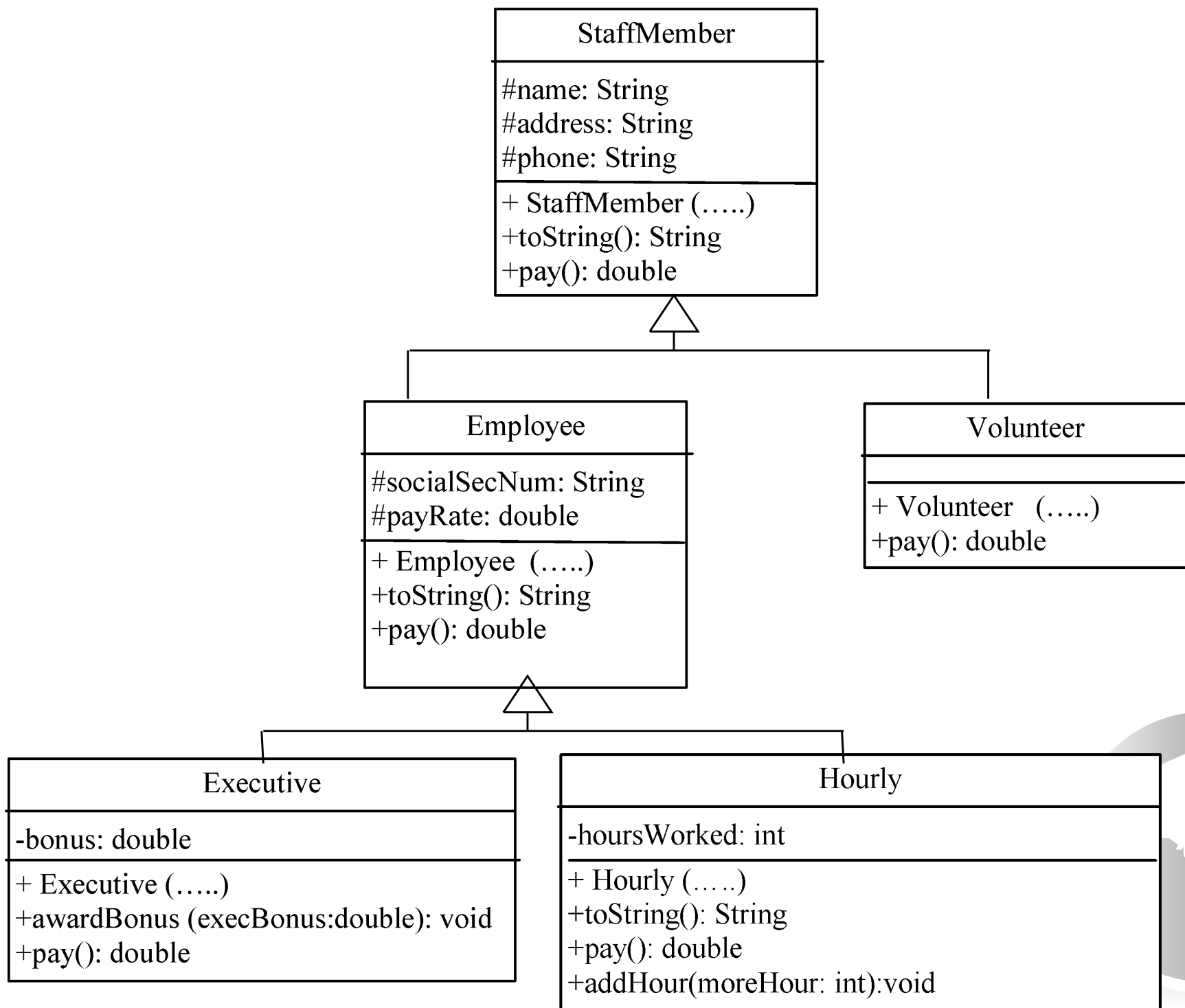
- 5 Groups of three.
- Implement each class according to the UML and the inheritance relationship.
- Each group will implement a class.



Polymorphism Example

- Consider the following class hierarchy:





```
public class StaffMember
```

```
{  
    protected String name;  
    protected String address;  
    protected String phone;
```

```
//-----
```

```
    public StaffMember (String eName, String eAddress, String ePhone)
```

```
{  
    name = eName;    address = eAddress;    phone = ePhone;  
}
```

```
//-----
```

```
    public String toString() {  
        String result = "Name: " + name + "\n";  
        result += "Address: " + address + "\n";  
        result += "Phone: " + phone;  
        return result;  
    }
```

```
//-----
```

```
    public double pay() {return 0;}  
}
```

StaffMember
#name: String #address: String #phone: String
+ StaffMember (.....) + toString(): String + pay(): double

Volunteer
+ Volunteer (.....)
+pay(): double

```

public class Volunteer extends StaffMember
{
    public Volunteer (String eName, String eAddress, String ePhone)
    {
        super (eName, eAddress, ePhone);
    }
    //-----
    public double pay() { return 0.0; }
}

```



```
public class Employee extends StaffMember
```

```
{
```

```
    protected String socialSecNum;
```

```
    protected double payRate;
```

```
//-----
```

```
    public Employee (String eName, String eAddress, String ePhone, String socSecNumber,  
                    double rate){
```

```
        super (eName, eAddress, ePhone);
```

```
        socialSecurityNumber = socSecNumber;
```

```
        payRate = rate;
```

```
    }
```

```
//-----
```

```
    public String toString()
```

```
{
```

```
        String result = super.toString();
```

```
        result += "\nSocial Security Number: " + socialSecurityNumber;
```

```
        return result;
```

```
    }
```

```
//-----
```

```
    public double pay() { return payRate; }
```

```
}
```

Employee
#socialSecNum: String #payRate: double
+ Employee (.....) +toString(): String +pay(): double

```
public class Executive extends Employee
```

```
{
```

```
    private double bonus;
```

```
    //-----
```

Executive
-bonus: double
+ Executive (.....)
+awardBonus (execBonus:double): void
+pay(): double

```
    public Executive (String eName, String eAddress, String ePhone, String socSecNumber,  
                      double rate) {
```

```
        super (eName, eAddress, ePhone, socSecNumber, rate);
```

```
        bonus = 0; // bonus has yet to be awarded
```

```
    }
```

```
    //-----
```

```
    public void awardBonus (double execBonus)
```

```
    {    bonus = execBonus; }
```

```
    //-----
```

```
    public double pay()
```

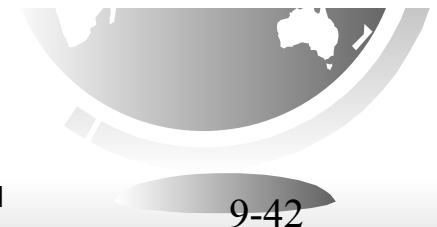
```
    {    double payment = super.pay() + bonus;
```

```
        bonus = 0;
```

```
        return payment;
```

```
    }
```

```
}
```



```
public class Hourly extends Employee
{
```

```
    private int hoursWorked;
```

```
//-----
```

Hourly
-hoursWorked: int
+ Hourly (.....)
+toString(): String
+pay(): double
+addHour(moreHour: int):void

```
    public Hourly (String eName, String eAddress, String ePhone, String socSecNumber,
                  double rate) {
```

```
        super (eName, eAddress, ePhone, socSecNumber, rate);
```

```
        hoursWorked = 0;
```

```
    }
```

```
//-----
```

```
    public void addHours (int moreHours) {    hoursWorked += moreHours; }
```

```
//-----
```

```
    public double pay() {
```

```
        double payment = payRate * hoursWorked;
```

```
        hoursWorked = 0;
```

```
        return payment;
```

```
    }
```

```
//-----
```

```
    public String toString() {
```

```
        String result = super.toString();
```

```
        result += "\nCurrent hours: " + hoursWorked;
```

```
        return result;
```

```

public class Staff {
    private StaffMember[] staffList;
    //-----
    public Staff ()
    {
        staffList = new StaffMember[6];
        staffList[0] = new Executive ("Sam", "123 Main Line", "555-0469", "123-45-6789", 2423.07);
        staffList[1] = new Employee ("Carla", "456 Off Line", "555-0101", "987-65-4321", 1246.15);
        staffList[2] = new Employee ("Woody", "789 Off Rocker", "555-0000", "010-20-3040", 1169.23);
        staffList[3] = new Hourly ("Diane", "678 Fifth Ave.", "555-0690", "958-47-3625", 10.55);
        staffList[4] = new Volunteer ("Norm", "987 Suds Blvd.", "555-8374");
        staffList[5] = new Volunteer ("Cliff", "321 Duds Lane", "555-7282");
    }
    //-----
    public void payday () {
        double amount;
        for (int count=0; count < staffList.length; count++)
        {
            System.out.println (staffList[count]);
            amount = staffList[count].pay(); // polymorphic
            if (amount == 0.0)    System.out.println ("Thanks!");
            else                System.out.println ("Paid: " + amount);
        }
    }
}

```

```
public class Firm
{
    //-----
    //  Creates a staff of employees for a firm and pays them.
    //-----
    public static void main (String[] args)
    {
        Staff personnel = new Staff();

        personnel.payday();
    }
}
```





Did the hourly employee receive any salary? Why? What do we need to do?

Did the Executive employee get bonus? What do we do?



```
public class Firm
{
    //-----
    //  Creates a staff of employees for a firm and pays them.
    //-----
    public static void main (String[] args)
    {
        Staff personnel = new Staff();

        personnel.addHours (40);
        personnel.addBouns (10000);
        personnel.payday();

    }
}
```

```

public class Staff {
    private StaffMember[] staffList;
    //-----
    public Staff ()
    {
        staffList = new StaffMember[6];
        staffList[0] = new Executive ("Sam", "123 Main Line", "555-0469", "123-45-6789", 2423.07);
        staffList[1] = new Employee ("Carla", "456 Off Line", "555-0101", "987-65-4321", 1246.15);
        staffList[2] = new Employee ("Woody", "789 Off Rocker", "555-0000", "010-20-3040", 1169.23);
        staffList[3] = new Hourly ("Diane", "678 Fifth Ave.", "555-0690", "958-47-3625", 10.55);
        staffList[4] = new Volunteer ("Norm", "987 Suds Blvd.", "555-8374");
        staffList[5] = new Volunteer ("Cliff", "321 Duds Lane", "555-7282");
    }
    //-----

```

```

    public void addHours ( int hour)
    {
        staffList[3].addHours (hour);
        ((Hourly)staffList[3]).addHours (hour);
    }
    //-----
    public void addBouns ( double bonus)
    {
        staffList[0].awardBonus (bouns);
        ((Executive)staffList[0]).awardBonus (bouns);
    }
    .....
}

```

a staffMember object is not necessarily an Hourly or Executive object. StaffMember class does not have addHours or addBouns methods.

Even though you can see that staffList[3] is really an Hourly object, the compiler is not so clever to know it.

To tell the compiler that staffList[3] is an Hourly object, use an explicit casting.

Legal assignments

```
class Test {  
    public static void main(String args[]) {  
        double d;  
        int i;  
  
        d = 5;           ✓  
        i = 3.5;         ✗  
        i = (int) 3.5;   ✓  
    }  
}
```

Since **double** is bigger data type than **int**, you can simply downcast **double** to **int** in **Java**. **double** is 64-bit primitive value and when you cast it to 32-bit **integer**, anything after the decimal point is lost.

- Widening is legal
- Narrowing is illegal (unless you cast)



Casting Objects

You have already used the casting operator to convert variables of one primitive type to another. *Casting* can also be used to convert an object of one class type to another within an inheritance hierarchy.

```
m(new Student());
```

```
public static void m(Object x)
```

```
Object x = new Student();
```

The statement is known as implicit casting, is legal because an instance of Student is automatically an instance of Object.





```
m(new Object());
```

```
public static void m(Student x) X
```

```
Student x = new Object();
```

Why does the statement **Object x = new Student()** work,
but the statement **Student x = new Object()** doesn't?

This is because a Student object is always an instance of
Object, but an Object is not necessarily an instance of Student.





When Casting is Necessary?

```
m(new Student());
```

```
public static void m(Object x) → Object x = new student ();
```

Suppose you want to assign the object reference x to a variable of the Student type using the following statement:

```
Student a = x;      X
```

Even though you can see that x is really a Student object, the compiler is not so clever to know it.

What do we do?

Use an explicit casting.

```
Student a = (Student) x;
```





You may also consider the analogy of fruit, apple, and orange with the Fruit class as the superclass for Apple and Orange.

```
Fruit fruit= new Apple();
```

```
Apple x = fruit;
```

```
Apple x = (Apple)fruit;
```

```
Orange x = (Orange)fruit;
```





You may also consider the analogy of fruit, apple, and orange with the Fruit class as the superclass for Apple and Orange.

`Fruit fruit= new Apple();` An apple is a fruit,

`Apple x = fruit;` **X** a fruit is not necessarily an apple

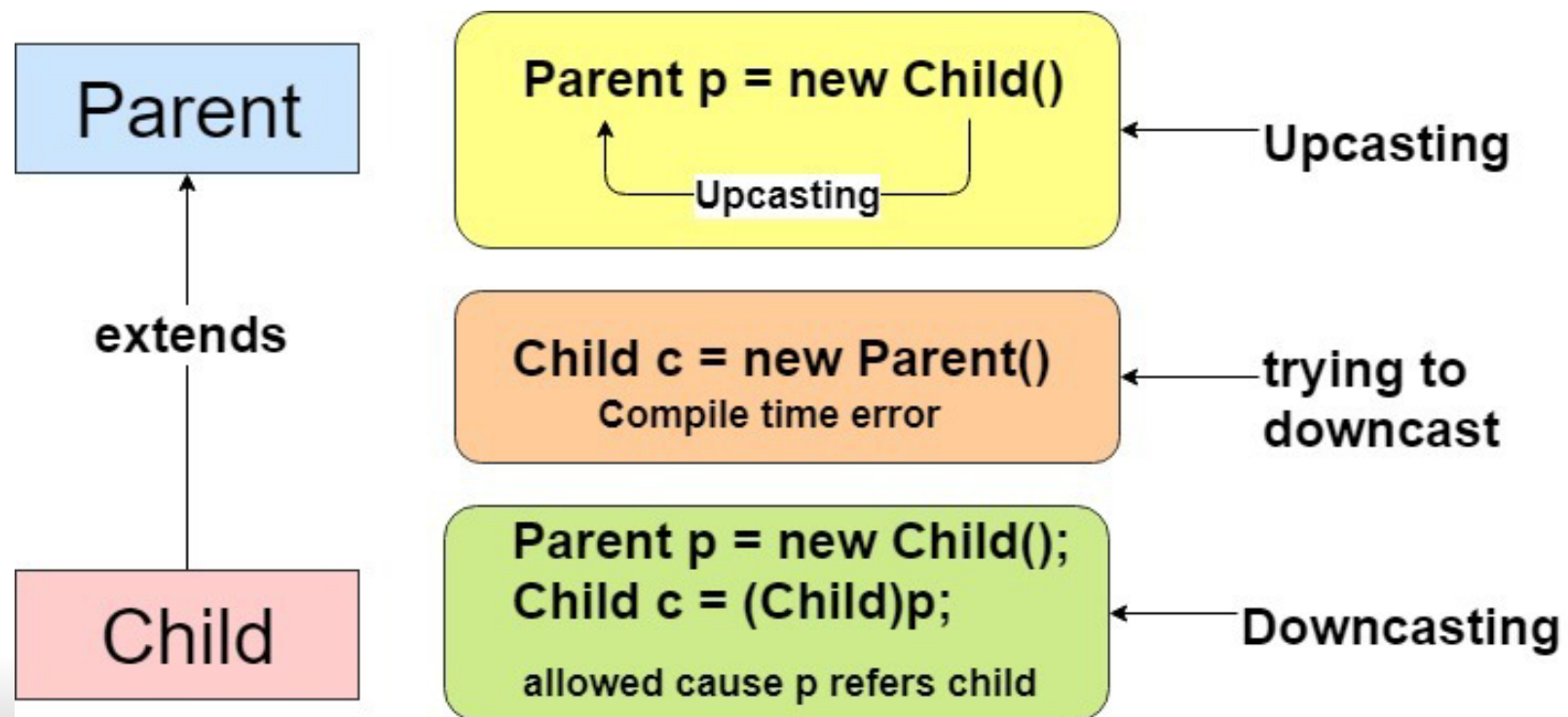
`Apple x = (Apple)fruit;` **✓** so you have to use explicit casting to assign an instance of Fruit to a variable of Apple

`Orange x = (Orange)fruit;` **X** Exception



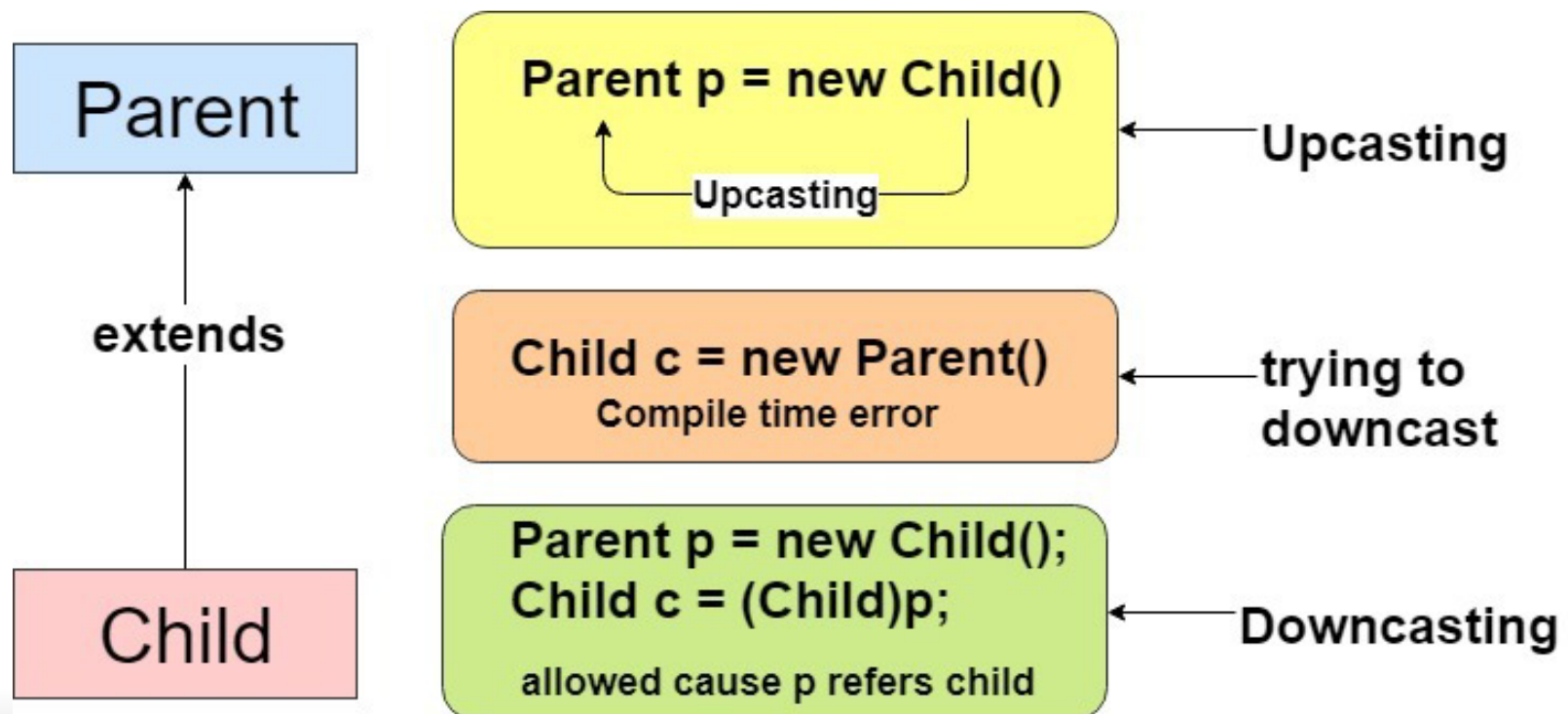
Upcasting vs. Downcasting

- It is always possible to cast an instance of a subclass to a variable of a superclass (known as upcasting), because an instance of a subclass is always an instance of its superclass.



Upcasting vs. Downcasting

- When casting an instance of a superclass to a variable of its subclass (known as downcasting), explicit casting must be used to confirm your intention to the compiler with the (SubclassName) cast notation.



Casting from Superclass to Subclass

Explicit casting must be used when casting an object from a superclass to a subclass. This type of casting may not always succeed.



Casting from Superclass to Subclass

- For the casting to be successful, you must make sure that **the object to be cast is an instance of the subclass.**
- If **the superclass object is not an instance of the subclass**, a **runtime ClassCastException** occurs.
- For example, if an object is not an instance of Student, it cannot be cast into a variable of Student.
- It is a good practice, therefore, to **ensure that the object is an instance of another object before attempting a casting.**
- This can be accomplished using the **instanceof** operator.



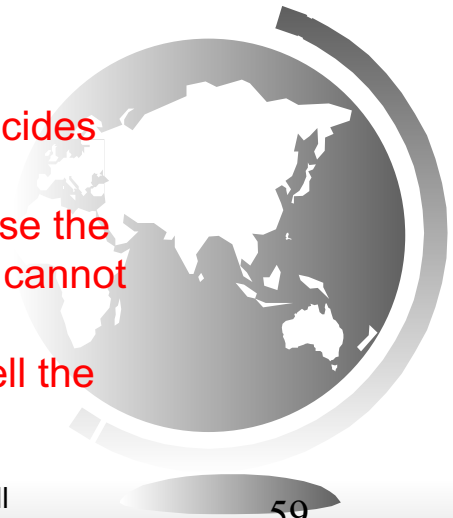
The instanceof Operator

Use the `instanceof` operator to test whether an object is an instance of a class:

```
Object myObject = new Circle();  
... // Some lines of code  
/** Perform casting if myObject is an instance of  
    Circle */  
if (myObject instanceof Circle) {  
    System.out.println("The circle diameter is " +  
        ((Circle)myObject).getDiameter());  
}
```

■ ■ ■

- The variable `myObject` is declared as `Object`. The declared type decides which method to match at compile time.
- Using `myObject.getDiameter()` would cause a compile error, because the `Object` class does not have the `getDiameter` method. The compiler cannot find a match for `myObject.getDiameter()`.
- Therefore, it is necessary to cast `myObject` into the `Circle` type to tell the compiler that `myObject` is also an instance of `Circle`.





```
public class Staff {  
    .....  
    public void addHours ( int hour)  
    {        ((Hourly)staffList[3]).addHours (hour);    }  
    //-----  
    public void addBouns ( double bonus)  
    {    ((Executive)staffList[0]).awardBonus (bouns); }  
    .....  
}
```

We don't know that staffList[3] is an Hour object , Or
staffList[0] is an Executive object...

So, what do we do?

Use the **instanceof** operator



The instanceof Operator

Use the `instanceof` operator to test whether an object is an instance of a class:

```
public void addHours ( int hour)
{
    /** Perform casting if s is an instance of Hourly*/
    for(StaffMember s: staffList)
        if (s instanceof Hourly)
            ((Hourly)s).addHours (hour);
}
//-----
public void addBouns ( double bonus)
{
    for(int i=0; i< staffList.length; ++i)
        if (staffList[i] instanceof Executive)
            ((Executive)staffList[i]).awardBonus (bouns);
}
```





Example: Demonstrating Polymorphism and Casting

The program creates two objects: a circle, and a rectangle, then invokes the `displayObject` method to display the objects. The `displayObject` displays the diameter if the object is a circle and displays the area if the object is a rectangle.



CastingDemo

Run



```
public class CastingDemo {
```

```
    public static void main(String[] args) {
```

```
        // Create and initialize two objects
```

```
        Object object1 = new Circle (1);
```

```
        Object object2 = new Rectangle (1, 1);
```

```
        // Display circle and rectangle
```

```
        displayObject(object1);
```

```
        displayObject(object2);
```

```
    }
```

```
    /** A method for displaying an object */
```

```
    public static void displayObject(Object object) {
```

```
        if (object instanceof Circle) {
```

```
            System.out.println("The circle diameter is " +  
                                ((Circle)object).getDiameter());
```

```
        }
```

```
        else if (object instanceof Rectangle) {
```

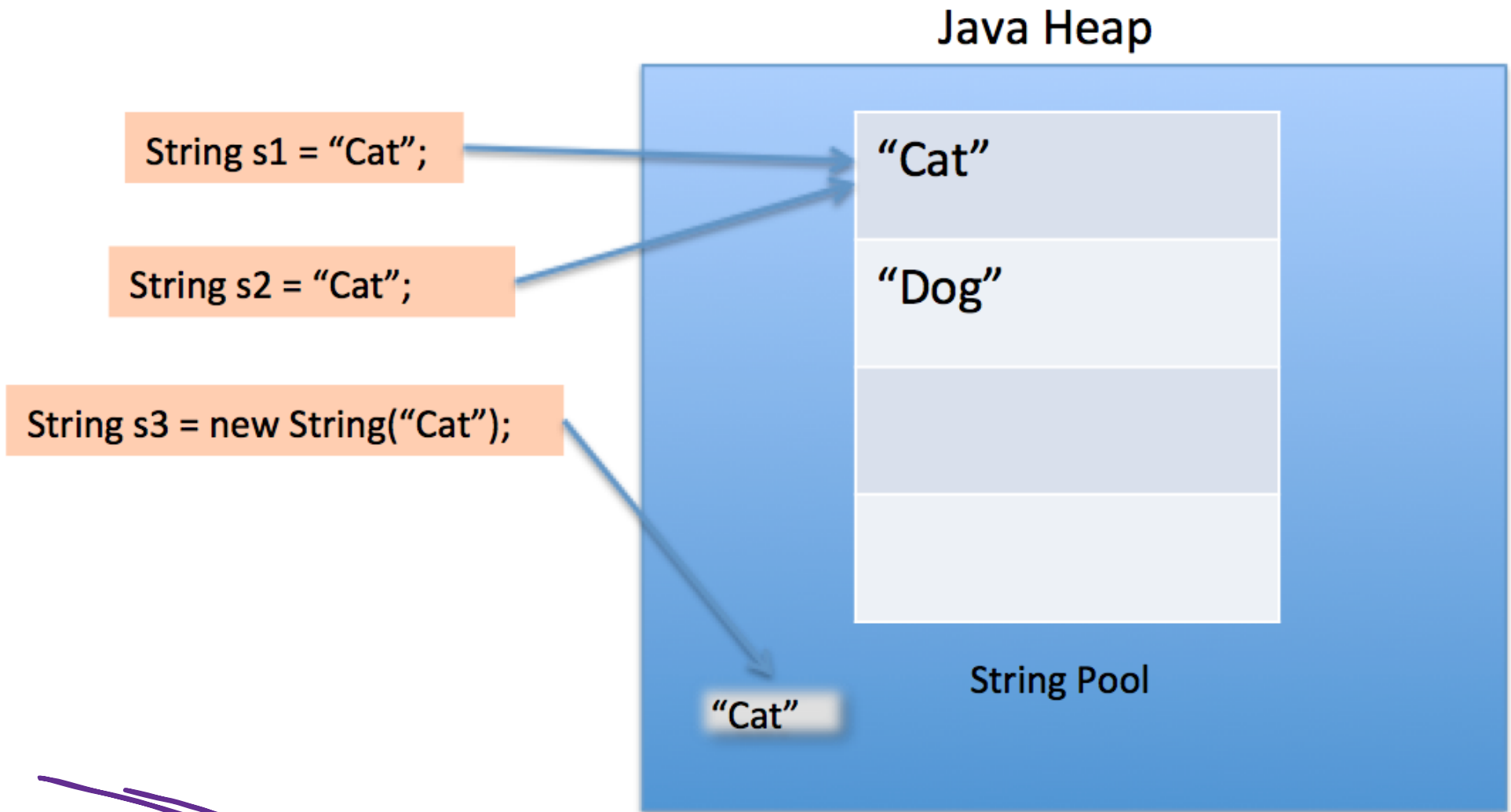
```
            System.out.println("The rectangle area is " +  
                                ((Rectangle)object).getArea());
```

```
        }
```

```
    }
```

```
}
```







Equality

- Consider these two assignments:

```
Thing thing1 = new Thing();
```

```
Thing thing2 = new Thing();
```

Are these two “Things” equal?

- That’s up to the programmer!

- Consider:

```
Thing thing3 = new Thing();
```

```
Thing thing4 = thing3;
```

Are these two “Things” equal?

- Yes, because they are the **same** Thing!



== operand

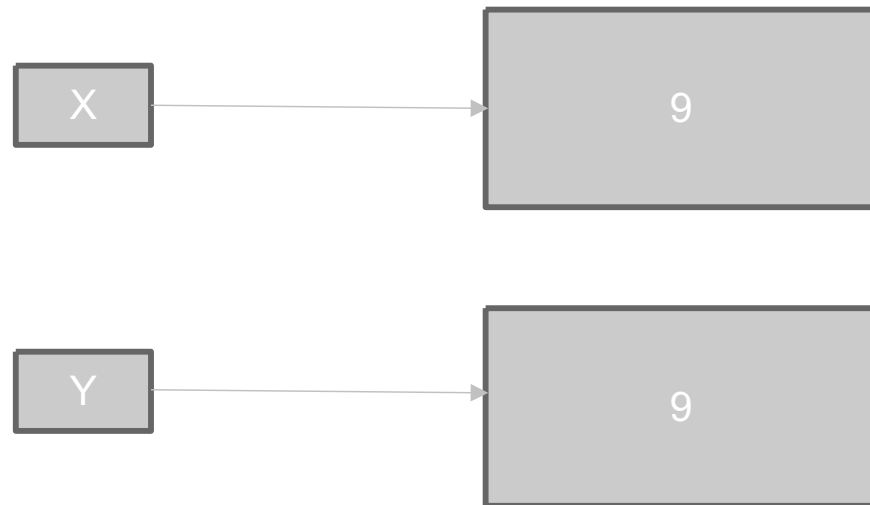
```
int x=9;  
int y=9;  
if (x==y)
```

- Primitives can always be tested for equality with ==



== operand

Integer x = new Integer (9); or String x = new String("abc");
Integer y = new Integer (9); or String y = new String("abc");
if(x==y) ? No

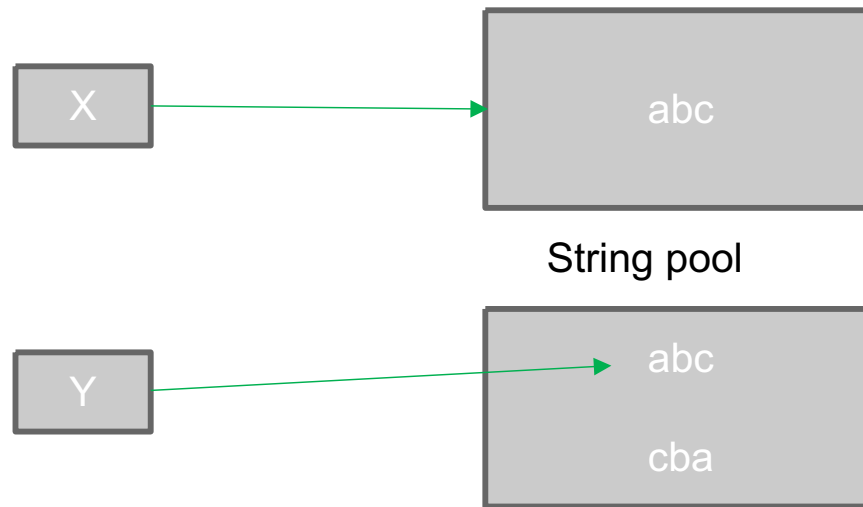


- For objects, == tests whether the two are the **same** object.
- Two Integers or Strings *may or may not be* == !



== operand

Integer x = new Integer (9); or String x =new String("abc");
Integer y = 9; or String y ="abc";
if(x==y) ? No

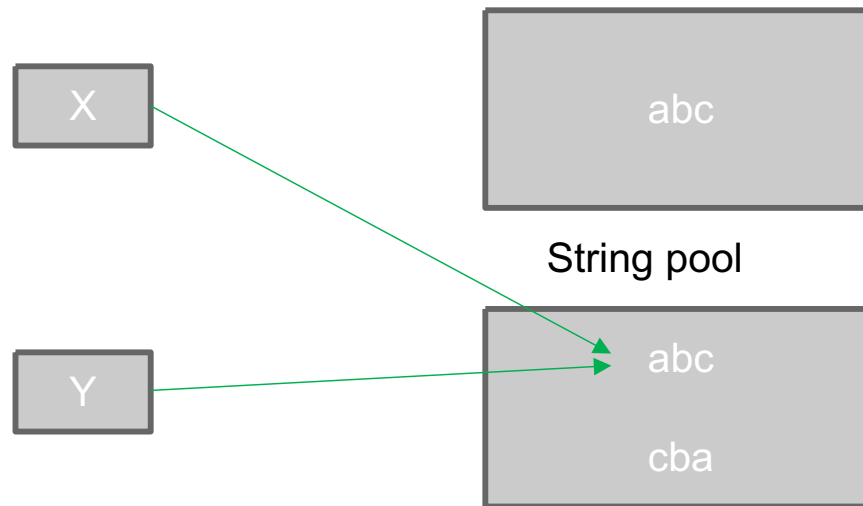


- For objects, == tests whether the two are the **same** object
- Two Integers or Strings *may or may not be* == !



== operand

Integer x = 9 or String x = "abc";
Integer y = 9; or String y = "abc";
if(x==y) ? Yes



- For objects, == tests whether the two are the **same** object
- Two Integers or Strings *may or may not be* == !



```
public class test{  
    public static void main(String[] args)  {  
  
        Integer x1= new Integer (9);  
        String x2=new String("abc");  
  
        Integer y1= 9;  
        Integer z1= 9;  
        String y2="abc";  
        String z2="abc";  
  
        System.out.print(x1==y1);  
        System.out.print(x2==y2);  
        System.out.print(y1==z1);  
        System.out.print(y2==z2);  
  
    }  
}
```



```

public class test{
    public static void main(String[] args)  {

        Integer x1= new Integer (9);
        String x2=new String("abc");

        Integer y1= 9;
        Integer z1= 9;
        String y2="abc";
        String z2="abc";

        System.out.print(x1==y1);
        System.out.print(x2==y2);
        System.out.print(y1==z1);
        System.out.print(y2==z2);

    }

```

false
 false
 true
 true



The equals method

- Objects can be tested with the method
`public boolean equals(Object o)` in Object Class.

Obj1.equals(Obj2);

What does it do?

The default implementation of the equals method in the Object class is as follows:

```
public boolean equals(Object obj) {  
    return this == obj;  
}
```

This implementation checks whether two reference variables point to the same object using the `==` operator.



The equals method

- The `equals ( )` method compares the contents of two objects.
- You should **override** this method in your custom class to test whether two distinct objects have the same content.
- The equals method is overridden in many classes in the Java API, such as **`java.lang.String`** and **`java.util.Date`**, to compare whether the contents of two objects are equal.



The equals method

Obj1 and Obj2 are Strings,

Obj1.equals(Obj2);

What does it do?

- It is overridden in the class `String`

Obj1 and Obj2 are arrays,

Obj1.equals(Obj2);

What does it do?

- It is *not* overridden for arrays; `==` tests if its operands are the *same* array.



Morals

- Never use `==` to test *equality* of Strings or arrays or other objects.
- Use `equals` for Strings, `java.util.Arrays.equals(a1, a2)` for arrays.
- If you test your own objects for equality, override `equals`.

For example, the `equals` method is overridden in the `Circle` class.

```
public boolean equals(Object o) {  
    if (o instanceof Circle) {  
        return radius == ((Circle)o).radius;  
    }  
    else  
        return false;  
}
```



```

public class Test {
public static void main(String[] args) {

                                public boolean equals(Object obj) {
Object circle1 = new Circle();    return this == obj;
Object circle2 = new Circle(); }
System.out.println(circle1.equals(circle2));
}
}
(a) class Circle {
double radius;
public boolean equals(Circle circle)
{ return this.radius == circle.radius; }
}
(b) class Circle {
double radius;
public boolean equals(Object o)
{ return this.radius == ((Circle)o).radius; }
}

```



```
public class Test { public static void
```

```
main(String[] args) {
```

```
    public boolean equals(Object obj) {
```

```
        return this == obj;
```

```
    }
```

```
Object circle1 = new Circle();
```

```
Object circle2 = new Circle();
```

```
System.out.println(circle1.equals(circle2)); }
```

```
}
```

```
(a) class Circle {
```

```
double radius;
```

```
public boolean equals(Circle circle)
```

```
{ return this.radius == circle.radius; }
```

```
}
```

```
(b) class Circle {
```

```
double radius;
```

```
public boolean equals(Object o)
```

```
{ return this.radius == ((Circle)o).radius; }
```

```
}
```

Overload

Override

The output is **false** if the Circle class in (a) is used.

The Circle class has two overloaded methods: **equals(Circle circle)** defined in the Circle class and **equals(Object o)** defined in the Object class, inherited by the Circle class. At compile time, `circle1.equals(circle2)` is matched to `equals(Object o)`, because the declared type for `circle1` and `circle2` is **Object**. (Note that either the declared type for `circle1` and `circle2` is `Object` would cause `circle1.equals(circle2)` to match `circle1.equals(Object circle)` by the compiler.

In (a), method `equals(Object o)` is used at both compile time and run time. `circle1` and `circle2` have different addresses, leading to "false" output. **Method overriding follows dynamic binding (determined by the actual type), but method overloading is always determined by the declared type.**

The output is **true** if the Circle class in (b) is used.

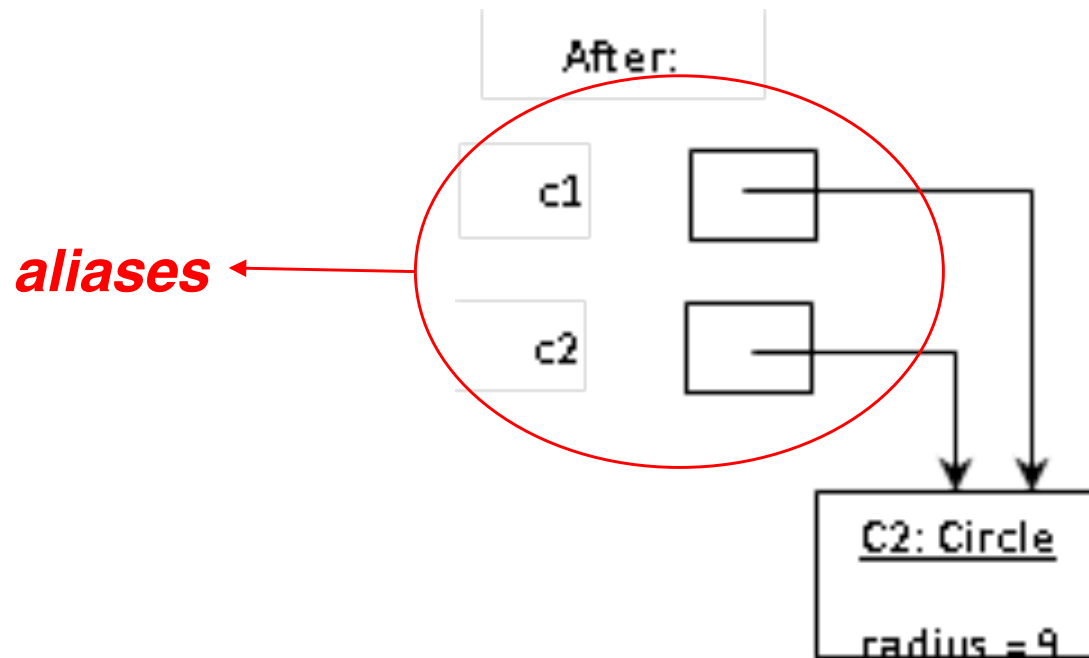
The Circle class overrides the `equals(Object o)` method defined in the Object class. At compile time, `circle1.equals(circle2)` is matched to `equals(Object o)` and at runtime the `equals(Object o)` method implemented in the Circle class is invoked.



NOTE

==	equals
is used for comparing two primitive data type values or for determining whether two objects have the same references.	is intended to test whether two objects have the same contents, provided that the method is modified in the defining class of the objects.
The == operator is stronger than the equals method, in that the == operator checks whether the two reference variables refer to the same object.	

Aliases

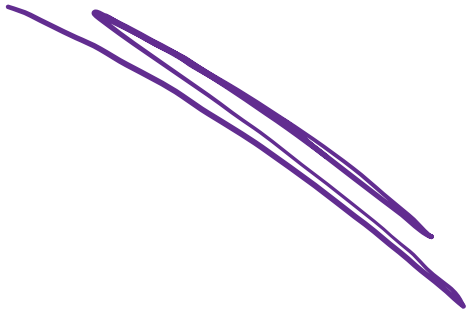


- Two or more references that refer to the same object are called *aliases* of each other.
- One object can be accessed using multiple reference variables (**C1 and C2**).



Aliases

- Aliases can be useful but should be managed carefully.
- Changing an object through one reference changes it for all of its aliases, because there is really only one object. (this doesn't work for Strings)





- What do you do if you want to save 100 student's records?

`Student[] array = new Students[100];`

You can create an array to store objects. But the array's size is fixed once the array is created.

- What if the number of students increases?

`copy the array to bigger one`

- What if you don't know the size?

`ArrayList class;`



The ArrayList Class

Java provides the ArrayList class that can be used to store an unlimited number of objects.

Creating and adding contents:

java.util.ArrayList<E>

```
+ArrayList()  
+add(o: E) : void  
+add(index: int, o: E) : void  
+set(index: int, o: E) : E
```

Creates an empty list.

Appends a new element *o* at the end of this list.

Adds a new element *o* at the specified index in this list.

Sets the element at the specified index. Return and remove the current one.



Differences and Similarities between Arrays and ArrayList

Operation	Array	ArrayList
Creating an array/ArrayList	<code>String[] a = new String[10]</code>	<code>ArrayList<String> list = new ArrayList<>();</code>
Adding a new element	<code>a[size-1] = "London";</code>	<code>list.add("London");</code>
Inserting a new element	Needs shifting procedures <code>a[index] = "London";</code>	<code>list.add(index, "London");</code>
Updating an element	<code>a[index] = "London";</code>	<code>list.set(index, "London");</code>



The ArrayList Class

Removing contents and checking status:

java.util.ArrayList<E>

```
+clear(): void  
+remove(o: Object): boolean  
+remove(index: int): boolean  
  
+contains(o: Object): boolean  
+isEmpty(): boolean  
+size(): int
```

Removes all the elements from this list.

Removes the element *o* from this list.

Removes the element at the specified index.

Returns true if this list contains the element *o*.

Returns true if this list contains no elements.

Returns the number of elements in this list.

Operation	Array	ArrayList
Removing an element	Needs shifting procedures	list.remove(index);
Removing an element		list.remove(Object);
Removing all elements	a = null;	list.clear();
Returning size	a.length	list.size();

The ArrayList Class

Searching for contents:

java.util.ArrayList<E>

```
+get(index: int): E  
+indexOf(o: Object): int  
+lastIndexOf(o: Object): int
```

Returns the element from this list at the specified index.

Returns the index of the first matching element in this list.

Returns the index of the last matching element in this list.

Operation	Array	ArrayList
Accessing an element	a[index]	list.get(index);

Differences and Similarities between Arrays and ArrayList

<i>Operation</i>	<i>Array</i>	<i>ArrayList</i>
Creating an array/ArrayList	<code>String[] a = new String[10]</code>	<code>ArrayList<String> list = new ArrayList<>();</code>
Accessing an element	<code>a[index]</code>	<code>list.get(index);</code>
Updating an element	<code>a[index] = "London";</code>	<code>list.set(index, "London");</code>
Returning size	<code>a.length</code>	<code>list.size();</code>
Adding a new element		<code>list.add("London");</code>
Inserting a new element		<code>list.add(index, "London");</code>
Removing an element		<code>list.remove(index);</code>
Removing an element		<code>list.remove(Object);</code>
Removing all elements		<code>list.clear();</code>



DistinctNumbers

Run

Array Vs. ArrayList

- `String [] s = new String [10];`
 - `s[0] = “One”; s[1] = “two”; ....`

This is a specific kind of array with particular size.

- `ArrayList s = new ArrayList ();`
- `ArrayList <E> s = new ArrayList<E> ();`
 - `s.add(“One”); s.add(“two”); ....`

Use generic ArrayList.

<E> is called generic type. You can replace it with concrete object type



Create ArrayList

- `ArrayList s = new ArrayList ();`
- `ArrayList <E> s = new ArrayList<E> ();`
 - `s.add(“One”); s.add(“two”); ....`

Use generic ArrayList.

<E> is called generic type. You can replace it with concrete object type.

ArrayList is known as a generic class with a generic type E. You can specify a concrete type to replace E when creating an ArrayList.



Generic Type

You can specify a concrete type to replace E when creating an ArrayList.

For example,

This ArrayList object can be used to store strings.

```
ArrayList<String> cities = new ArrayList<String>();
```

```
ArrayList<String> cities = new ArrayList<>();
```



TestArrayList

Run



```
ArrayList s = new ArrayList();  
s.add("one");  
String n = s.get(0);
```

ERROR → Requires Casting

```
ArrayList s = new ArrayList();  
s.add("one");  
String n = (String) s.get(0);
```

SOLUTION → Explicit Casting

```
ArrayList <String> s = new ArrayList<String>();  
s.add("one");  
String n = s.get(0);
```

SOLUTION → Concrete Type



```
public class TestArrayList
{
public static void main(String[] args)
{
```

```
    // Create a list to store cities
```

```
    ArrayList<String> cityList = new ArrayList<>();
```

```
    // Add some cities in the list
```

```
    cityList.add("London");
```

```
    // cityList now contains [London]
```

```
    cityList.add("Denver");
```

```
    // cityList now contains [London, Denver]
```

```
    cityList.add("Paris");
```

```
    // cityList now contains [London, Denver, Paris]
```

```
    cityList.add("Miami");
```

```
    // cityList now contains [London, Denver, Paris, Miami]
```

```
    cityList.add("Seoul");
```

```
    // contains [London, Denver, Paris, Miami, Seoul]
```

```
    cityList.add("Tokyo");
```

```
    // contains [London, Denver, Paris, Miami, Seoul, Tokyo]
```



```
System.out.println("List size? " + cityList.size());
System.out.println("Is Miami in the list? " + cityList.contains("Miami"));
System.out.println("The location of Denver in the list? " +
    cityList.indexOf("Denver"));
System.out.println("Is the list empty? " + cityList.isEmpty());
// Print false
// Insert a new city at index 2
cityList.add(2, "Xian");
// contains [London, Denver, Xian, Paris, Miami, Seoul, Tokyo]
// set a city at index 3
cityList.set(3, "Jeddah");
// contains [London, Denver, Xian, Jeddah, Miami, Seoul, Tokyo]
// Remove a city from the list
cityList.remove("Miami");
// contains [London, Denver, Xian, Jeddah, Seoul, Tokyo]
// Remove a city at index 1
cityList.remove(1);
// contains [London, Xian, Jeddah, Seoul, Tokyo]
// Display the contents in the list
System.out.println(cityList.toString());
```



```
// Display the contents in the list in reverse order
```

```
for (int i = cityList.size() - 1; i >= 0; i--)  
System.out.print(cityList.get(i) + " ");  
System.out.println();
```

```
// Create a list to store two circles
```

```
java.util.ArrayList<Circle> list = new java.util.ArrayList<>();
```

```
// Add two circles
```

```
list.add(new Circle (2));  
list.add(new Circle(3));
```

```
// Display the area of the first circle in the list
```

```
System.out.println("The area of the circle? " + list.get(0)  
.getArea());  
}  
}
```





The MyStack Classes



MyStack

- Implement each Method

MyStack	
-list: ArrayList	A list to store elements.
+isEmpty(): boolean	Returns true if this stack is empty.
+getSize(): int	Returns the number of elements in this stack.
+peek(): Object	Returns the top element in this stack.
+pop(): Object	Returns and removes the top element in this stack.
+push(o: Object): void	Adds a new element to the top of this stack.
+search(o: Object): int	Returns the position of the first element in the stack from the top that matches the specified element.

```

import java.util.ArrayList;
public class MyStack
{
    private ArrayList<Object> list = new ArrayList<>();
    public boolean isEmpty()
    { return list.isEmpty(); }
    public int getSize()
    { return list.size(); }
    public Object peek()
    { return list.get(getSize() - 1); }
    public Object pop()
    {
        Object o = list.get(getSize() - 1);
        list.remove(getSize() - 1);
        return o;
    }
    public void push(Object o)
    { list.add(o); }
    @Override /** Override the toString in the Object class */
    public String toString()
    { return "stack: " + list.toString(); }
}

```





Array Lists from/to Arrays

We can convert primitive type to objects using Wrapper classes.
Can we convert ArrayList to array and vice versa?

YES

Using the Static method “asList” in the Arrays Class, we can convert an array to an ArrayList.

```
String[] array = {"red", "green", "blue"};
```

```
ArrayList<String> list = new ArrayList<>(Arrays.asList(array));
```





Array Lists from/to Arrays

We can convert primitive type to objects using Wrapper classes.
Can we convert ArrayList to array and vice versa?

YES

Using the method “toArray” in the ArrayList, we can convert an ArrayList to an array.

```
String[] array1 = new String[list.size()];
```

```
list.toArray(array1);
```





If you want to sort, find max/min, or shuffle an array, what do you do?

Use an Algorithm that needs many statements and looping.

Are there other easier solutions?

Yes. Use the static methods “sort”, “max”, “min” and “Shuffle” in the `java.util.Collections` class, but the objects in the `ArrayList` must be comparable (Wrappers and Strings) objects to perform the tasks.



sort,max,min and shuffle in an Array List

```
Integer[] array = {5,8,2,9,4};
```

```
ArrayList<Integer> list = new ArrayList<>(Arrays.asList(array));
```

```
Collections.sort(list);
```

```
System.out.println(list);
```

[2, 4, 5, 8, 9]

```
System.out.println(Collections.max(list));
```

9

```
System.out.println(Collections.min(list));
```

2

```
Collections.shuffle(list);
```

```
System.out.println(list);
```

[4, 5, 9, 2, 8]

